Lembar Ringkas Alpro · Minggu 1–6

Algoritma dan Pemrograman (BBK1AAB4) · Python 3 · Baca dari kiri-atas ke kanan-bawah: tiap minggu memakai minggu sebelumnya.

W01 Dari masalah ke program

COMPUTATIONAL THINKING

EMPAT CARA BERPIKIR

Dekomposisi
Pecah masalah besar jadi bagian yang bisa diselesaikan dan diuji terpisah.

Pengenalan pola
Temukan bagian yang berulang; selesaikan sekali, pakai ulang.

Abstraksi
Abaikan detail yang tidak relevan, supaya satu solusi berlaku untuk banyak kasus.

Berpikir algoritmik
Susun langkah-langkah berurutan yang menyelesaikan masalah.

ALGORITMA → PROGRAM

Algoritma = urutan langkah yang **tidak ambigu**, **terbatas**, dan **konsisten** (masukan sama → hasil sama). **Program** = algoritma yang ditulis dalam bahasa yang bisa dijalankan komputer. Rancang dulu, baru tulis: tentukan **INPUT → PROSES → OUTPUT**.

```
BACA panjang, lebar
luas ← panjang × lebar
TULIS luas
```

```
panjang = float(input())
lebar = float(input())
luas = panjang * lebar
print(luas)
```



PYTHON DIJALANKAN OLEH INTERPRETER

Kompiler menerjemahkan seluruh program sekali sebelum dijalankan. **Interpreter** (Python) menerjemahkan dan menjalankan baris demi baris.

Jenis galat	Kapan terlihat	Contoh
Sintaksis	sebelum jalan; aturan tulis dilanggar	`print("Halo"`
Runtime	program berhenti saat jalan	`10 / 0` → ZeroDivisionError
Semantik	jalan, tapi hasilnya salah	`rata = a + b / 2`

OPERATOR ARITMETIKA

Op	Arti	Contoh	Op	Arti	Contoh
`+`	tambah	`7 + 2` → 9	`/`	bagi (selalu float)	`7 / 2` → 3.5
`-`	kurang	`7 - 2` → 5	`//`	bagi, bulat ke bawah	`7 // 2` → 3
`*`	kali	`7 * 2` → 14	`%`	sisa bagi	`7 % 2` → 1
`**`	pangkat	`2 ** 3` → 8	`8 / 2` → 4.0 (tetap float)		

Urutan: `( )` → `**` → `* / // %` → `+ -`; sederajat dari kiri ke kanan. `2 + 3 * 4 ** 2` → `2 + 3 * 16` → **50**

JEBAKAN `-7 // 2` → **-4** (dibulatkan ke bawah, bukan dipotong). Ganjil/genap: `n % 2 == 0` berarti genap.

W02 Variabel, tipe data, masukan

U2.1–U2.5

PENUGASAN: LABEL PADA NILAI

`harga = 12000` → tempelkan nama **harga** pada nilai 12000. Python menghitung **sisi kanan dulu**, lalu menempelkan nama di kiri. `x = x + 1` memindahkan **x** ke nilai baru. Nama yang belum diberi nilai → **NameError**.

```
a = 78
b = 91
a = b      # 78 hilang!
b = a
```

baris	a	b
1	78	–
2	78	91
3	91	91
4	91	91

Trace table: satu baris per pernyataan, satu kolom per variabel. Menukar: `sementara = a` dulu.

ATURAN NAMA VARIABEL

Diawali huruf atau `_`; hanya huruf, angka, `_`; tanpa spasi; bukan keyword; huruf besar ≠ kecil (`Nilai` ≠ `nilai`). Pakai gaya `nama_jelas`.

✔ `nilai_akhir` `nilai_2` `_total` ✘ `2nilai` `nilai akhir` `harga-1` `class`

Ekspresi menyusut jadi satu nilai: `harga * 3 + 500` → 36500. **Statement** adalah perintah yang mengubah keadaan: `total = 0`.

EMPAT TIPE DASAR

Tipe	Isi	Contoh
`int`	bilangan bulat	`3` `-7` `0`
`float`	bilangan desimal	`3.5` `2.0`
`str`	teks, dalam tanda kutip	`"Bandung"` `"7"`
`bool`	nilai kebenaran	`True` `False`

Tipe melekat pada **nilai**, bukan nama: `type(7.0)` → `<class 'float'>`.

Operasi	Hasil	Operasi	Hasil
`3 + 4`	7 (int)	`"ab" + "cd"`	`"abcd"`
`3 + 4.0`	7.0 (float)	`"ha" * 3`	`"hahaha"`
`6 / 3`	2.0 (float)	`"7" + 1`	TypeError

MASUKAN, KONVERSI, KELUARAN

```
jumlah_teks = input("Berapa tiket? ")  # "3" → str
jumlah = int(jumlah_teks)              # 3   → int
print(f"Total: {jumlah * 25000}")      # Total: 75000
print("Nilai:", 85)                    # Nilai: 85
```

`int(3.9)` → 3 · `int(-3.9)` → -3 · `int("12")` → 12

`float("12.5")` → 12.5 · `str(7)` → `"7"` · `int("12.5")` → ValueError

JEBAKAN `input()` **selalu** menghasilkan teks. `"3" + "4"` → `"34"`, bukan 7. Ubah dulu dengan `int()` atau `float()`. `int()` memotong ke arah nol, tidak membulatkan.

CATATANKU · tulis sendiri: hal yang masih sering kamu salahkan

W03 Kondisional & guard

U3.1–U3.5

EKSPRESI BOOLEAN → TRUE / FALSE

`==` sama · `!=` tidak sama · `<` `<=` `>` `>=`. Dengan `nilai = 75`: `nilai >= 70` → True, `nilai != 75` → False.

`and` True jika **kedua** sisi True · `or` True jika **minimal satu** sisi True · `not` membalik. **Urutan:** perbandingan → `not` → `and` → `or`. `=` menempelkan nama; `==` bertanya.

JEBAKAN `if hari == "Sabtu" or "Minggu":` selalu True (teks dianggap True). Tulis kondisi lengkap di tiap sisi: `hari == "Sabtu" or hari == "Minggu"`. Begitu juga `x > 0 and x < 10`, bukan `x > 0 and < 10`.

IF · IF-ELSE · IF-ELIF-ELSE

```
if nilai >= 80:
    huruf = "A"
elif nilai >= 70:
    huruf = "B"
else:
    huruf = "C"
print(huruf)    # di luar blok
```

Diperiksa **dari atas**; berhenti di kondisi **True pertama**; tepat satu cabang dijalankan.

Indentasi menentukan baris mana yang ikut sebuah cabang.

Dua `if` terpisah = dua keputusan; `elif` = bagian keputusan yang sama.

Urutkan rentang dari yang paling ketat: `>= 60` di atas membuat 95 dapat C.

BERSARANG: KEPUTUSAN DI DALAM KEPUTUSAN

`if` dalam `if` hanya diperiksa bila kondisi luarnya True. Setiap `else` berpasangan dengan `if` pada **indentasi yang sama**. Tanpa `else`, bisa diratakan:

```
if punya_tiket:
    if umur >= 13:
        print("Masuk")
```

```
if punya_tiket and umur >= 13:
    print("Masuk")
```

elif berantai: satu variabel, rentang tak tumpang tindih · **bersarang**: pertanyaan kedua bergantung pada yang pertama.

GUARD: PERIKSA MASUKAN SEBELUM DIPROSES

```
umur_teks = input()                  # BACA
if not umur_teks.isdigit():          # GUARD
    print("Harus angka bulat")
else:
    umur = int(umur_teks)            # UBAH
    if umur > 120:                   # TOLAK
        print("Tidak masuk akal")
    elif umur >= 17:                 # CEK SYARAT
        print("Boleh membuat KTP")
    else:
        print("Belum cukup umur")
```

Pola: BACA → GUARD → UBAH → TOLAK → PILIH KATEGORI → CEK SYARAT, tiap langkah di dalam `else` langkah sebelumnya. `"20".isdigit()` True · `"-3".isdigit()` False.

Lembar Ringkas Alpro · Minggu 1–6

Fungsi, nilai kembalian, dan perulangan · Tutup lembar ini, coba jawab sendiri dulu, baru cek di sini.

W04 Fungsi, parameter, lingkup

U4.1–U4.5

ANATOMI FUNGSI

```
def cetak_tagihan(nama, total):          # header
    """Mencetak tagihan satu pelanggan."""  # docstring
    print(f"{nama}: Rp{total}")          # badan

cetak_tagihan("Ani", 45000)              # posisi
cetak_tagihan(total=45000, nama="Ani") # kata kunci
```

`def` hanya **mencatat** fungsi; badannya jalan saat **dipanggil** (tanpa pemanggilan, tidak ada keluaran). **Parameter** = nama di kurung header; **argumen** = nilai yang dikirim, dipasangkan menurut **urutan** (atau menurut nama, untuk argumen kata kunci). `help(cetak_tagihan)` menampilkan docstring.

STACK DIAGRAM

Setiap pemanggilan mendapat **frame** sendiri berisi parameter dan variabel lokalnya. Frame hilang saat fungsi selesai.

```
__main__
(definisi)
cetak_tagihan → fungsi
```

```
cetak_tagihan
nama → "Ani"
total → 45000
```

Traceback dibaca dari baris **terakhir** (jenis galat) ke atas (baris pemicunya).

FUNGSI VOID DAN NONE

Fungsi tanpa `return` melakukan aksi, tetapi hasilnya `None`.

```
hasil = cetak_tagihan("Ani", 0)    # mencetak Ani: Rp0
print(hasil)                        # None
```

LINGKUP (SCOPE)

Variabel di dalam fungsi bersifat **lokal**: tidak terlihat di luar (→ **NameError**).

```
skor = 0
def tambah_skor():
    skor = skor + 1    # UnboundLocalError
```

Nilai masuk lewat **parameter**, keluar lewat `return` (W05), bukan lewat variabel global.

ENKAPSULASI → GENERALISASI

Enkapsulasi: kode berulang → satu fungsi. **Generalisasi**: nilai tertanam → parameter: `cetak_garis()` → `cetak_garis(panjang)`. **Dekomposisi**: satu fungsi, satu tugas.

KAMUS PESAN GALAT · W03–W04

Penyebab	Pesan yang muncul
`if nilai = 75:`	SyntaxError … Maybe you meant '=='?
`if nilai >= 60` (lupa titik dua)	SyntaxError: expected ':'
indentasi tidak rata	IndentationError
`int("dua")`	ValueError: invalid literal for int()
argumen kurang dari parameter	TypeError … missing 1 required positional argument

W05 Fungsi ber-return

U5.1–U5.5

RETURN VS PRINT

	`return`	`print`
Ke mana?	kembali ke pemanggil	ke layar
Bisa dihitung lagi?	ya, dipakai di ekspresi	tidak; hasil fungsi `None`
Sesudahnya	fungsi langsung selesai	fungsi berlanjut

```
def ongkir(berat):
    """Ongkos kirim Rp10.000 per kg."""
    return berat * 10000

total = 50000 + ongkir(2)    # 70000
print(ongkir(3) == 30000)    # True (baris uji)
```

BEBERAPA JALUR RETURN

```
def tarif_layanan(layanan):
    if layanan == "kilat":
        return 9000
    elif layanan == "reguler":
        return 6000
    return 0           # tanpa ini → None
    print("selesai")   # kode mati
```

Setiap jalur harus bertemu `return`; jalur tanpa `return` menghasilkan `None`. Baris sesudah `return` tidak pernah dijalankan.

FUNGSI BOOLEAN

```
def dapat_diskon(belanja):
    return belanja >= 100000
```

```
if dapat_diskon(150000):
    print("Diskon!")
```

Kembalikan ekspresinya langsung; tidak perlu `if … return True else return False` atau `== True`.

KOMPOSISI FUNGSI

Hasil satu fungsi dipakai di fungsi lain:

```
biaya = tarif_layanan(layanan) * berat - potongan(berat, layanan)
```

RESEP 5 LANGKAH

1. **Pahami**: tulis ulang soal; tentukan input, output, dan satu contoh hitungan tangan.
2. **Pecah**: satu sub-tugas = satu fungsi.
3. **Rancang**: header + docstring + `return 0` + satu baris uji.
4. **Isi & uji**: satu fungsi sampai ujinya `True`, baru lanjut. Uji juga **nilai batas** (tepat 4 kg pada aturan "lebih dari 4 kg").
5. **Rakit** bagian utama: `input()` → panggil fungsi → `print()`.

JEBAKAN Mencetak ≠ mengembalikan. Jika `ongkir` hanya `print(berat * 10000)` tanpa `return`, maka `ongkir(2) + 5000` → **TypeError** (None + int).

CATATANKU · contoh soal yang ingin kamu ingat

W06 Perulangan for & while

U6.1–U6.5

FOR DENGAN RANGE: BATAS TIDAK IKUT

`range(…)`	nilai variabel loop	`range(…)`	nilai
`range(4)`	0 1 2 3	`range(2, 11, 3)`	2 5 8
`range(1, 4)`	1 2 3	`range(5, 0, -2)`	5 3 1
`range(1, n + 1)`	1 … n	`range(5, 0)`	0 putaran

Sesudah loop, variabel loop menyimpan nilai **terakhir**: setelah `for k in range(2, 11, 3)`, `k` = 8.

AKUMULATOR & PENCACAH DI DALAM FUNGSI

```
def total_tabungan(minggu):
    """Setoran minggu ke-k = k × 10000."""
    total = 0              # sebelum loop
    for k in range(1, minggu + 1):
        total = total + k * 10000
    return total           # sejajar for

print(total_tabungan(3))   # 60000
```

k	total
–	0
1	10000
2	30000
3	60000

Akumulator: `total = total + x` · **Pencacah**: `banyak = banyak + 1` (biasanya di dalam `if`). Penampung diberi nilai awal **sebelum** loop; `return` **sesudah** loop.

WHILE: KONDISI DIPERIKSA SEBELUM TIAP PUTARAN

① **inisialisasi** sebelum loop · ② **kondisi** · ③ **proses** · ④ **pemutakhiran** di akhir badan. Bisa berjalan **0 kali**. Tentukan kapan **berhenti**, lalu balik: "berhenti saat -1" → `while hujan != -1:`

```
def total_hujan():
    """Total hujan sah; -1 selesai, -9 rusak."""
    total = 0
    hujan = int(input())       # ① baca nilai pertama
    while hujan != -1:         # ② sentinel tidak diproses
        if hujan != -9:        # ③ guard + akumulasi
            total = total + hujan
        hujan = int(input())   # ④ baca berikutnya, sejajar if
    return total               # ⑤

print(total_hujan())  # 12 -9 30 0 -1 → 42
```

Rata-rata: dua penampung, lalu jaga pembagian: `if banyak > 0:` `return total / banyak`, lainnya `return 0`.

for: banyak putaran diketahui sebelum loop · **while**: berhenti karena nilai yang datang (data diakhiri -1).

JEBAKAN **Infinite loop**: pemutakhiran lupa, atau `input()` berikutnya masuk ke dalam `if`. **Off-by-one**: `hari = 0` lalu `while hari <= 7` → 8 putaran. Tanpa baca nilai pertama, -1 ikut dijumlah.

KAMUS PESAN GALAT · W06

`for i in 5:`	TypeError: 'int' object is not iterable → `range(5)`
`total` tanpa nilai awal	UnboundLocalError
`input()` setelah data habis	EOFError: sentinel tidak pernah dibaca